

Assignment 2

Program - IoT, Robotics and Drones

Name – Abhishek Tandon



DATE : JULY 4, 2026

Q1. Arduino UNO vs ESP32 for a Smart Irrigation System

You are building a smart irrigation system that must monitor soil moisture, control a water pump, and send data to a mobile application every 5 seconds. You have both an Arduino UNO and an ESP32. Which controller would you choose and why? Would your answer change if internet connectivity was not required?

Ans:

Comparison

Parameter	Arduino UNO	ESP32
Processor	8-bit ATmega328P @ 16 MHz	32-bit dual-core Xtensa LX6 @ up to 240 MHz
RAM	2 KB	~520 KB
Flash	32 KB	4 MB (typical dev board)
Connectivity	None on-board (needs ESP8266/HC-05 shield)	Wi-Fi + BLE built in
Multitasking	Single-threaded super-loop only	FreeRTOS, dual-core task split possible
Cost (board)	₹250–300 (bare board)	₹450–600 (Wi-Fi included)
Active power draw	~45–50 mA, limited native sleep support	~80–160 mA active, ~10 µA deep sleep

On paper the UNO looks cheaper, but that comparison is misleading: to get the same Wi-Fi capability on an UNO you would add an ESP8266 module, a level shifter (UNO is 5 V logic, ESP8266 is 3.3 V), and extra wiring effort. Once that is priced in, the UNO-plus-Wi-Fi-module system usually costs about the same as a single ESP32 board while being harder to debug and less reliable at the connector interfaces.

The 5-second update requirement also matters: the loop needs to read an analog moisture value, decide on pump state, drive a relay, and simultaneously maintain a Wi-Fi/HTTP or MQTT session without blocking the sensor read. On a single-core UNO this becomes a timing headache; on the ESP32, one core can be dedicated to networking (via FreeRTOS tasks) while the other handles the deterministic sensor-and-actuator loop.

If internet connectivity was not required

The answer shifts, but only partially. For a purely standalone unit (read moisture → drive pump → optional local LCD/buzzer, no app), an Arduino UNO is perfectly adequate and is actually the more cost-effective and easier-to-teach choice for a bare-bones prototype, since the whole logic is a simple threshold-and-actuate loop with no concurrency pressure. I would still lean towards ESP32 if the project is expected to grow (data logging to SD, future

BLE configuration, OTA firmware fixes) because retrofitting connectivity onto a UNO-based design later is far more painful than starting on ESP32 and simply not using its radio.

Q2. Communication Strategy for 500 Sensors over 2 km (Smart Factory)

A smart factory contains 500 sensors distributed across a 2 km area. Compare Wi-Fi, Bluetooth, LoRa, and MQTT-based communication. Which protocol or combination of protocols would you choose and why?

Ans:

It helps to first separate the two things being compared, because they are not on the same layer. Wi-Fi, Bluetooth and LoRa are physical/link-layer radio technologies; MQTT is an application-layer protocol that rides on top of an IP connection (usually Wi-Fi or Ethernet). So the real question is: which radio technology gets the data from 500 distributed nodes to a gateway, and MQTT is almost certainly the answer for the gateway-to-cloud leg regardless of what is chosen below.

Comparison Matrix

Criteria	Wi-Fi	Bluetooth (BLE)	LoRa
Range	30–50 m indoor, ~100 m open	10–30 m (mesh extends this)	2–5 km (line-of-sight, factory NLOS ~1–2 km)
Data rate	10–100+ Mbps	~1–2 Mbps	0.3–50 kbps
Power consumption	High (~80–200 mA active)	Low (~10–15 mA active)	Very low (~10–40 mA TX, µA sleep)
Scalability (500 nodes)	Poor without many APs — channel congestion	Moderate, mesh helps but hop-latency grows	Excellent — designed for thousands of nodes per gateway
Infrastructure cost	High — multiple APs + cabling for 2 km coverage	Moderate — mesh routers needed	Low — 1–2 gateways can cover 2 km
Fit for this scenario	Good only for a few high-bandwidth nodes (cameras)	Weak at 2 km without a dense mesh	Strong fit for range + node count

A smart factory containing 500 sensors spread across a two-kilometre area presents a challenging communication problem because the chosen technology must balance range, power consumption, scalability, reliability, and infrastructure cost.

Wi-Fi provides high data rates and is ideal for applications involving video streaming or large data transfers. However, its typical coverage range of 50 to 100 metres would require dozens of access points to cover the factory premises. Managing connections from 500 sensors would also create significant network congestion and increase power consumption, making Wi-Fi unsuitable as the primary communication technology.

Bluetooth performs better in terms of power efficiency but suffers from an even shorter range, typically between 10 and 100 metres depending on the version used. While Bluetooth Mesh networking can extend coverage, maintaining a stable mesh across 500 industrial sensors would introduce unnecessary complexity and latency.

LoRa, on the other hand, was specifically designed for long-range, low-power communication. A single LoRa gateway can often cover several kilometres, allowing the entire factory to operate using only a few gateways. Since industrial sensors generally transmit only small packets of data such as temperature, vibration, or pressure values, LoRa's lower data rate is not a significant limitation.

MQTT deserves special mention because it is not a physical communication technology but rather an application-layer messaging protocol based on the publish-subscribe model. MQTT minimizes bandwidth usage and allows efficient communication between gateways, cloud servers, and monitoring dashboards.

The most practical solution would therefore be a **hybrid architecture using LoRa and MQTT**. Individual sensors would transmit data to nearby LoRa gateways, which would then forward the information to a cloud server using MQTT over Ethernet or Wi-Fi. This architecture combines long range, low power consumption, and excellent scalability while keeping infrastructure costs manageable.

Such hybrid architectures are increasingly common in modern Industry 4.0 deployments because they effectively combine the strengths of multiple communication layers rather than relying on a single technology for every task.

Q3. Diagnosing an Impossible Sensor Reading (-120°C / 300°C)

A temperature sensor suddenly starts showing impossible values (e.g., -120°C or 300°C). How you would determine whether the problem lies in the sensor, wiring, power supply, ADC, or software.

Ans:

An out-of-physical-range reading almost never means the temperature is actually wrong — it means something upstream of the "temperature" number is broken. The efficient way to find the fault is to isolate the chain one link at a time, starting from the cheapest test (swap the sensor) and moving toward the least likely but hardest-to-check cause (software).

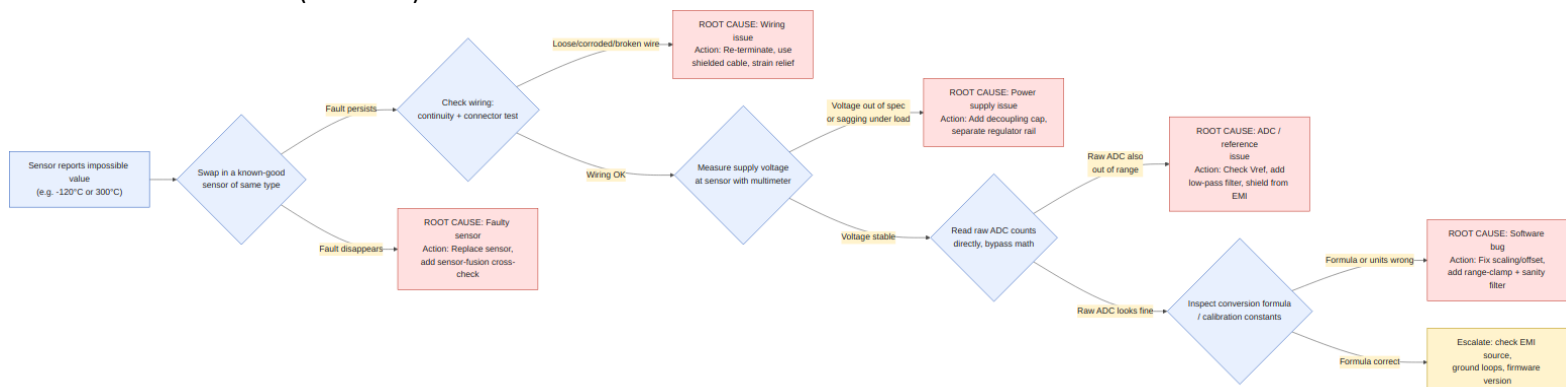


Figure Q3: Structured diagnostic flowchart for isolating sensor faults

If a temperature sensor suddenly begins reporting physically impossible values such as -120°C or 300°C , it is important to avoid immediately assuming that the sensor itself has failed. In embedded systems, incorrect readings can originate from multiple points in the measurement chain including the sensor, wiring, power supply, ADC subsystem, or software.

The investigation should begin by checking whether the abnormal values are constant or intermittent. Constant extreme values often indicate an open circuit, short circuit, or disconnected sensor, whereas random fluctuations are more commonly associated with power supply noise or software errors.

Reasoning behind the order

- **Sensor swap first** — it is the fastest single test: if a known-good sensor of the same part number immediately reads correctly in the same slot, the fault is confirmed to be the original sensor (common causes: internal short, moisture ingress, ESD damage).
- **Wiring next** — checked with a continuity/insulation test and visual inspection for corrosion, pinched cable, or a loose crimp; a broken or intermittent wire often produces exactly this kind of wild swing because the ADC input floats.
- **Power supply** — measured directly at the sensor pins under load; a sagging or noisy rail will corrupt an analog sensor's output reference and is easy to miss if you only check the regulator output at no load.
- **ADC / reference** — read the raw ADC count and compare against expected range before any scaling; if raw counts are already out of bounds, the fault is at the ADC/reference, not further downstream (common cause: floating V_{ref} , EMI coupling from a nearby motor or relay).
- **Software last** — only after hardware is cleared do we suspect a wrong scaling constant, sign error, or missing bounds check, since these are the hardest to physically verify and should not be the first guess.

Q4. Six-Month Battery Life for an ESP32 Weather Station

An ESP32-based weather station must operate on battery power for six months without maintenance. What hardware and software modifications would you make to maximize battery life?

Ans:

Operating an ESP32-based weather station continuously for six months on battery power requires aggressive power optimization at both the hardware and software levels because wireless communication and sensor operation dominate energy consumption.

Hardware changes

- **Deep sleep as the default state** — ESP32 deep sleep draws roughly $10\text{--}20\ \mu\text{A}$ versus $\sim 80\text{--}160\ \text{mA}$ active, so if the device is awake for only $\sim 1\text{--}2$ seconds per 10-minute cycle, average current can fall to tens of microamps.
- **Duty-cycled sensing** — sample every 10–15 minutes instead of continuously; weather does not change fast enough to need second-by-second logging.

- **Low-power sensor choice** — e.g., a BME280 (few μA in forced mode) instead of a continuously-powered industrial-grade probe.
- **Efficient regulator** — a low-quiescent-current buck/LDO (e.g., $\sim 1\text{--}5\ \mu\text{A}$ quiescent) instead of a generic linear regulator that itself burns milliamps at idle.
- **Remove/disable onboard LEDs and unused peripherals** on the dev board — power LEDs alone can draw several mA continuously.
- **Creative addition — small solar cell + supercapacitor buffer**: a coin-cell-sized solar panel trickle-charging a supercap in parallel with the main battery can offset self-discharge and extend runtime well past six months without adding much bulk, provided the enclosure allows light exposure.

Software changes

- **Batch and compress transmissions** — wake, sample, buffer a few readings, then send one payload rather than opening a fresh Wi-Fi/TCP session every cycle (the Wi-Fi association handshake is one of the most expensive events in the whole duty cycle).
- **Prefer ESP-NOW over full Wi-Fi/MQTT** where a nearby gateway exists — it skips DHCP/TCP overhead entirely and can cut radio-on time by more than half.
- **RTC timer wake-up** instead of polling in a loop, so the CPU is truly asleep between cycles.
- **Reduce TX power** to the minimum that still gives reliable link margin to the gateway/router.

Q5. Communication for a Near-Zero-Delay Automotive Braking System

An automatic braking system in a vehicle requires communication with almost zero delay. Which communication method would you choose and why?

Ans:

Braking is a hard real-time, safety-critical control loop, so the deciding factor is not raw bandwidth but determinism — the guarantee that a message will arrive within a bounded, predictable time even under worst-case bus load and electrical noise.

Option	Determinism	Latency	EMI robustness	Best use in a vehicle
CAN Bus	High — priority-based arbitration	$\sim 1\text{ ms}$ worst case for high-priority frames	Excellent (twisted pair, differential)	Brake, powertrain, safety-critical control
Automotive Ethernet	Moderate — needs TSN for hard guarantees	Low, but queuing jitter without TSN	Good (100BASE-T1 shielded)	High-bandwidth backbone: cameras, sensor fusion
Wi-Fi	Low — contention-based, unpredictable	Variable, can spike badly	Poor — susceptible to jamming/interference	Infotainment, non-critical telemetry only

Option	Determinism	Latency	EMI robustness	Best use in a vehicle
Bluetooth	Low	Variable, connection-dependent	Poor at vehicle-scale interference	Diagnostics dongles, phone pairing
SPI / UART	High but point-to-point only	Very low	Good over short traces	Local link: MCU to a single driver IC, not bus-wide

CAN Bus (or CAN-FD for more throughput) is the automotive-industry standard for exactly this reason: its bitwise arbitration means the highest-priority message (like a brake command) always wins the bus without waiting behind lower-priority traffic, worst-case latency is bounded and calculable, and the differential twisted-pair physical layer is extremely resistant to the electrical noise generated by motors, ignition systems, and switching relays in a vehicle.

Wi-Fi and Bluetooth are ruled out immediately — both are contention-based (CSMA), meaning latency depends on how busy the shared spectrum is, and both are vulnerable to interference or jamming, which is unacceptable for a system where a missed or delayed message can mean a missed brake command.

Automotive Ethernet is increasingly used for high-bandwidth needs (camera feeds for ADAS) but without Time-Sensitive Networking (TSN) extensions it does not natively guarantee the same bounded worst-case latency as CAN, and TSN adds cost and design complexity that is hard to justify for a single control loop.

SPI/UART are fine only for a short, local, point-to-point link (e.g., MCU to brake actuator driver IC) and do not scale to a vehicle-wide bus with multiple ECUs.

Q6. Optimal Sensor Placement in a Cuboid Classroom

You need to measure the average temperature of a classroom using only one sensor. Where would you place the sensor and why? Explain how poor placement could affect the accuracy of your measurements. Assume the classroom to be a square-base cuboid of side length “a” and height “b”.

Ans:

With only one sensor available, the goal is to find the single point whose reading is the best statistical representative of the whole room’s air, which means avoiding every local heat/cold source and picking a location where air is well-mixed.

Classroom Cross-Section: Sensor Placement (cuboid, side "a", height "b")

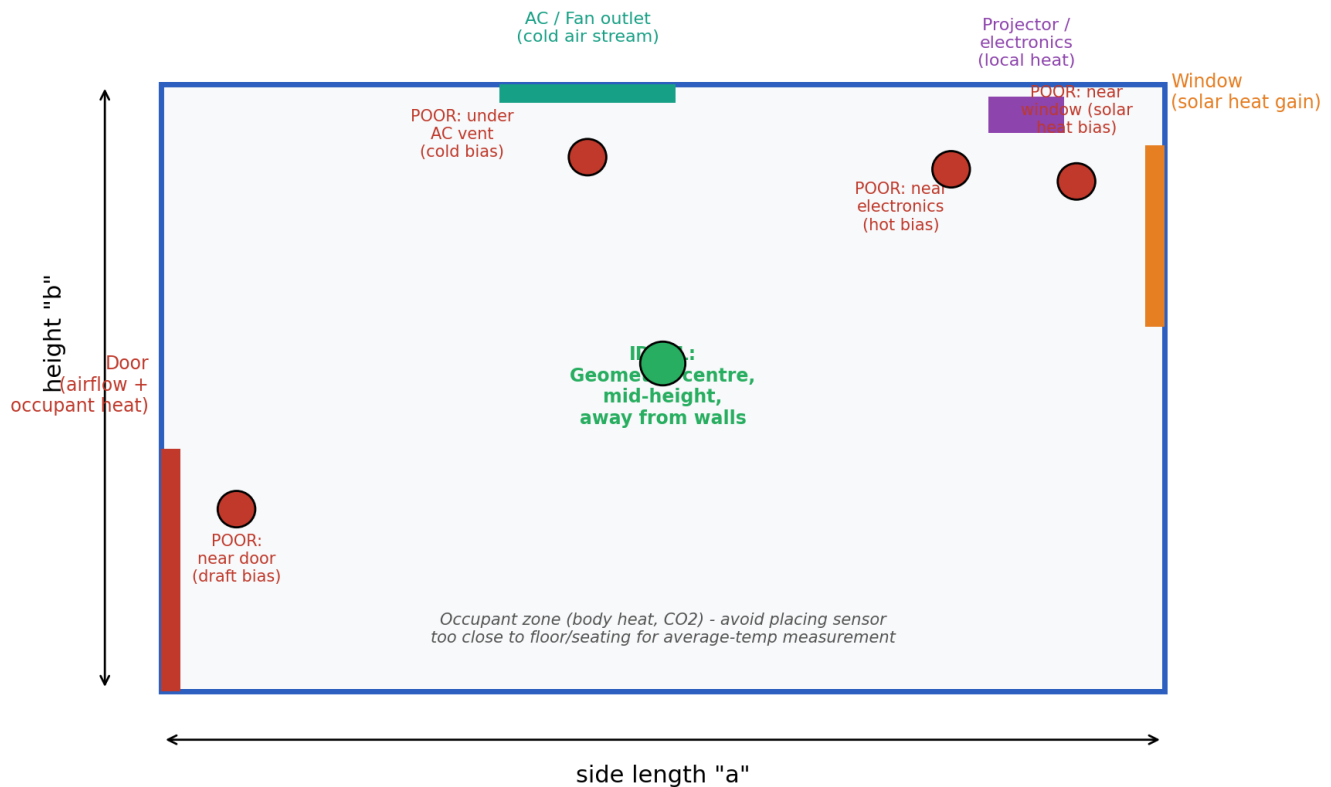


Figure Q6: Cross-section of a classroom (cuboid, side "a", height "b") showing ideal vs poor sensor locations

Placement logic

- **Ideal location** — close to the geometric centre of the room in plan, and at roughly mid-height (around $b/2$, i.e. approximate seated/standing breathing-zone height, $\sim 1.2\text{--}1.5\text{ m}$), away from any wall, door, or window by at least $0.5\text{--}1\text{ m}$. This position is farthest from all localized disturbances and sits in the zone where convective mixing averages out corners.
- **Near the door** — picks up draft from opening/closing and corridor air, biasing readings low in winter and creating noisy spikes correlated with occupant movement rather than the room's actual thermal state.
- **Near a window** — direct or indirect solar heat gain through glass causes a localized hot spot, especially in the afternoon, so a sensor here would overstate the room average significantly at certain times of day.
- **Directly under an AC/fan vent** — conditioned air blows straight onto the sensor, causing it to under-read relative to the actual occupied-zone temperature.
- **Near a projector or other electronics** — localized heat dissipation from equipment causes an upward bias unrelated to general room comfort.

The underlying physics reason all of these are "poor" is the same: a classroom is not thermally uniform — it has boundary layers near walls/windows (radiative and conductive effects) and localized plumes near heat/cold sources (convective effects). A single point sensor can only report the temperature at its exact location, so the

placement choice implicitly decides what "room temperature" even means. Placing it centrally and mid-height minimizes the influence of any one boundary effect and lets natural air mixing do the averaging for you, rather than the algorithm.

Q7. Situations Where Analog Sensors Outperform Digital Ones

Under what situations could an analog sensor perform better than a digital one?

Ans:

Digital sensors are usually preferred for their built-in calibration, noise-immunity via error checking, and ease of multiplexing several sensors on one bus — but there are specific situations where an analog sensor is genuinely the better engineering choice, not just a legacy option:

- **Very fast transient signals** : a digital sensor has an internal ADC-and-protocol conversion delay (I2C/SPI transaction time), which adds latency. A raw analog output (e.g., a photodiode or piezo vibration sensor feeding the MCU's own ADC directly) can be sampled as fast as the MCU's ADC allows, which matters for flame detection, fast pressure spikes, or knock sensing.
- **Long cable runs in electrically noisy environments** : industrial 4–20 mA current-loop transmitters are analog, and the reason they dominate factory floors over long distances is that a current signal is inherently immune to voltage-drop and induced-noise errors along a cable; a digital signal over the same 100+ m run would need RS-485, fibre, or repeaters to stay reliable.
- **Simple, single-variable, cost-sensitive designs** : a thermocouple with a simple op-amp signal-conditioning stage can be cheaper and simpler than a smart digital sensor plus its driver/firmware/library overhead, especially for a one-off measurement with no need for a shared bus.
- **Customized Measurement Systems** : Another advantage appears in customized measurement systems. Since the raw analog signal is directly available, engineers can implement their own amplification, filtering, calibration, and signal processing techniques tailored to the application. Digital sensors, on the other hand, often hide their internal processing algorithms from the user.
- **Avoiding bus contention and address conflicts** : if many sensors already share an I2C bus, adding one more digital device risks address clashes or bus-timing issues; routing that one extra signal through a spare ADC channel instead is often the pragmatic fix.
- **No firmware/driver dependency** : an analog sensor needs no library, no register configuration, and cannot "hang" a communication bus the way a misbehaving I2C/SPI digital sensor occasionally can.

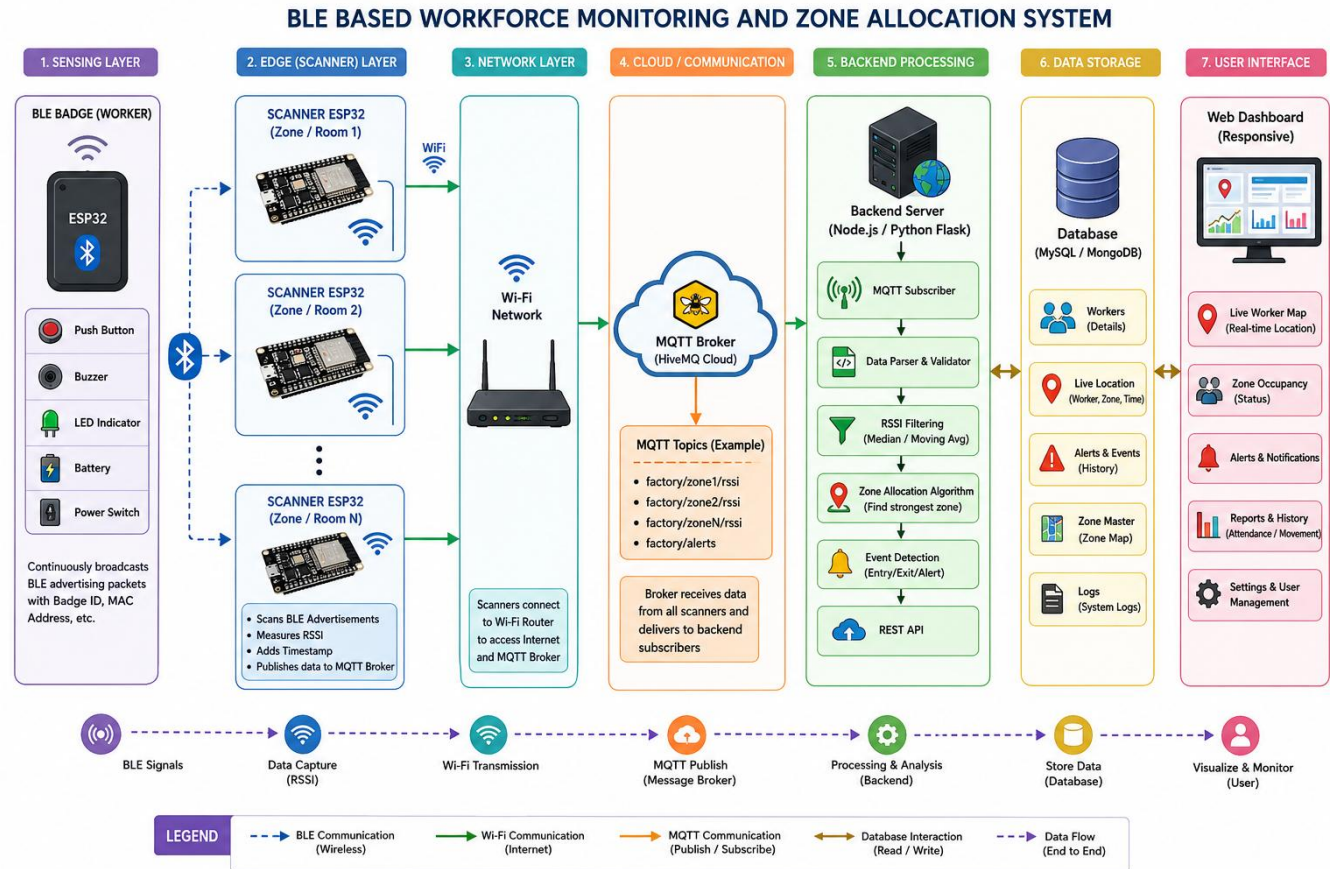
However, analog sensors are generally more susceptible to electromagnetic interference, cable losses, and ADC inaccuracies, making proper signal conditioning essential.

Therefore, analog sensors continue to play an important role in applications involving high-speed measurements, custom signal processing, low-cost deployments, and situations where access to raw sensor information is desirable.

Q8. End-to-End IoT Architecture with Failure-Point Analysis

Draw the complete architecture/block diagram of your IoT project, beginning from the sensor and ending at the user interface. Identify every possible point where a failure could occur and suggest one solution for each.

Ans:



1. Badge Battery Failure

Failure

Badge stops transmitting.

Effect

Worker disappears from system.

Solution

- Low battery monitoring.
- Buzzer warning.
- Rechargeable battery schedule.

2. ESP32 Badge Crash

Failure

Firmware hangs.

Effect

No BLE advertisements.

Solution

Use watchdog timer.

If no transmission for 10 seconds

→ ESP automatically restarts

3. BLE Packet Collision

Failure

Many badges transmit simultaneously.

Effect

Some packets lost.

Solution

- Random advertisement intervals.
- Multiple packet transmissions.

4. Scanner ESP Power Failure

Failure

Scanner turns off.

Effect

Entire zone becomes invisible.

Solution

- UPS backup.
- Health monitoring message every minute.

5. Scanner ESP Firmware Crash

Failure

ESP hangs.

Solution

Enable watchdog timer.

6. WiFi Disconnection

Failure

ESP cannot reach broker.

Effect

RSSI data lost.

Solution

- Automatic reconnect loop.
- Local buffering of messages.

7. MQTT Broker Failure

Failure

Cloud broker unavailable.

Effect

Entire system communication stops.

Solution

- Secondary backup broker.
- Local Mosquitto broker as fallback.

8. Internet Failure

Failure

Factory internet stops.

Effect

Cloud communication lost.

Solution

Deploy local broker inside factory LAN.

This is one of the strongest solutions for industrial deployment.

9. RSSI Fluctuation

Failure

Worker shown in wrong zone.

Example:

Actual zone = Room 2

Detected zone = Room 3

Cause

- Human body absorption
- Walls
- Multipath reflections
- Orientation changes

Solution

- Median filtering
- Hysteresis logic
- Multiple scanners per area

This is the biggest practical challenge of BLE localization.

10. Equal RSSI from Two Zones

Example:

Zone1 = -67

Zone2 = -68

System cannot decide.

Solution

Use hysteresis:

Change zone only if

new RSSI is stronger by 5 dBm

for 5 consecutive readings.

11. Database Failure

Failure

Database server crashes.

Effect

Historical data lost.

Solution

- Daily backups.
- Replication server.

12. Backend Server Failure

Failure

Website stops updating.

Solution

- Docker restart policy.
- Multiple backend instances.

13. Website Failure

Failure

Dashboard unavailable.

Solution

- Host on cloud with redundancy.
- Use health checks.

14. Time Synchronization Failure

Failure

Events stored with wrong timestamps.

Solution

Use NTP synchronization on all ESPs.

15. Unauthorized Device Injection

Failure

Fake badge sends packets.

Solution

- Whitelist MAC addresses.
- Authenticate devices.

16. MQTT Security Attack

Failure

Unauthorized publish or subscribe.

Solution

- Username/password authentication.
- TLS encryption.

17. Duplicate Badge IDs

Failure

Two workers appear as same person.

Solution

Use unique MAC address or UUID.

Complete Failure Table

Stage	Failure	Solution
Badge	Battery dead	Battery alerts
Badge	Firmware crash	Watchdog timer
BLE	Packet collision	Random intervals
Scanner	Power loss	UPS
Scanner	Crash	Watchdog
WiFi	Disconnection	Auto reconnect
MQTT	Broker down	Backup broker
Internet	Outage	Local broker
RSSI	Fluctuation	Filtering
Algorithm	Wrong zone	Hysteresis
Backend	Server crash	Redundancy
Database	Failure	Backup
Website	Down	Cloud redundancy
Security	Fake badge	Authentication

Stage	Failure	Solution
Time	Clock drift	NTP

Q9. Wi-Fi-Based Attack Surface and Practical Mitigations

If an attacker gains access to your IoT device through the Wi-Fi network, what kinds of attacks could they perform? Suggest at least five methods to improve the security of the system (with applicable solutions, do not cite far-fetched ideas).

Ans:

Once an attacker is on the same Wi-Fi network as an IoT device, the attack surface widens considerably because many IoT devices assume the local network is trusted. The table below lists realistic attacks (not exotic ones) and mitigations that are actually deployable on a resource-constrained device.

Attack	How it happens	Practical mitigation
Credential theft	Default/weak passwords, credentials sent in plaintext	Unique per-device credentials at provisioning, never ship default passwords, store creds in encrypted NVS
Packet sniffing	Unencrypted HTTP/MQTT traffic captured on the LAN	Enforce TLS/DTLS for all data in transit (HTTPS, MQTTS)
Replay attacks	Captured valid command replayed later (e.g., "unlock" message)	Timestamp + sequence number in every message, reject duplicates/out-of-window messages
Unauthorized firmware upload	Attacker pushes malicious firmware via an unsecured OTA endpoint	Signed firmware images verified by SHA-256 + digital signature before flashing (see Q17)
Denial of Service	Flooding the device or broker with requests	Rate limiting on the broker, network segmentation (dedicated IoT VLAN), connection limits per device
Device hijacking	Exploiting an open/unused service (e.g., default Telnet, debug port)	Disable all unused ports/services, minimal firmware attack surface, WPA2/WPA3-Enterprise instead of shared PSK

Several practical security improvements can significantly reduce these risks:

1. Use WPA2 or WPA3 protected Wi-Fi networks.
2. Encrypt all communication using TLS/HTTPS.
3. Store credentials in encrypted secure storage.
4. Enable secure boot and signed firmware updates.
5. Disable unnecessary network services and open ports.

6. Use strong authentication and password policies.
7. Implement firewall rules and network segmentation.
8. Regularly update firmware to patch vulnerabilities.

Security in IoT systems should be treated as a continuous process rather than a one-time configuration task.

Q10. Conflicts Interfacing DHT11, Ultrasonic, OLED, Relay, and Soil Moisture on One ESP32

Your ESP32 must simultaneously interface with a DHT11 sensor, an ultrasonic sensor, an OLED display, a relay module, and a soil moisture sensor. What hardware or software conflicts could occur, and how would you solve them?

Ans:

Putting five peripherals on one ESP32 mostly fails not because of a single big mistake, but because of several small, unrelated conflicts stacking up. Each is listed with its concrete fix below.

Conflict type	Specific issue	Fix
GPIO conflicts	Boot-strapping pins (GPIO0, 2, 15) misused for sensors can prevent boot; ADC2 pins conflict with active Wi-Fi	Reserve strapping pins for their boot role; read soil moisture on an ADC1 channel, not ADC2
Timing conflicts	DHT11 needs a strict ~20–25 ms bit-banged timing window	Isolate DHT11 read in its own FreeRTOS task/core so it is not delayed by other loop code
Blocking code	Ultrasonic pulseIn() and DHT11 read both block the main loop, starving OLED refresh and relay timing	Use interrupt-based echo-pulse timing (or a timer capture) instead of pulseIn(); split work across ESP32's two cores
Power supply issues	Relay coil switching causes voltage spikes/dips on the shared rail	Flyback diode across the relay coil, separate 5 V rail for the relay with common ground, decoupling capacitors near ESP32
ADC interference	Relay switching noise couples into the soil-moisture analog reading	RC low-pass filter on the ADC input; opto-isolate the relay driver from the sensing rail
Interrupt conflicts	Two peripherals assigned to the same interrupt-capable pin, or ISR too long	Assign each interrupt source a distinct pin, keep ISRs short (just flag + timestamp, process in the main loop)

If I had to prioritize a single architectural fix rather than patch each symptom individually, it would be to **move blocking, timing-sensitive reads (DHT11, ultrasonic) onto a dedicated FreeRTOS task pinned to one core**, leaving the other core free for OLED refresh, relay logic, and network activity. This turns five uncoordinated peripherals into two cleanly separated concurrency domains instead of one tangled super-loop.

Q11. Offline-First Architecture for a 12-Hour Connectivity Outage

An ESP32 uploads sensor data to the cloud every minute. The internet connection fails for 12 hours. Design a strategy that ensures no important data is permanently lost.

Ans:

The design goal is simple to state but requires care to implement correctly: **no reading taken during the outage should be permanently lost, and no reading should be uploaded twice.** That means every reading needs a durable local copy, a way to know if it was already delivered, and a deletion rule that only fires after confirmed delivery.

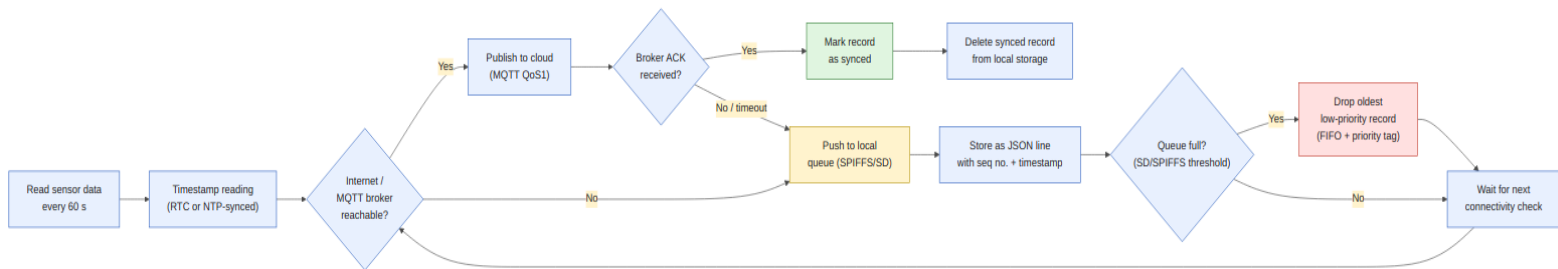


Figure Q11: Offline-first data flow with local buffering, retry, and acknowledgement-based deletion

Design elements

- **Storage medium** — SPIFFS is fine for short outages (12 hours of once-a-minute readings is only 720 small records), but SD card over SPI is the safer choice if outages could occasionally run much longer, since SPIFFS has limited total capacity and SD offers gigabytes of headroom at low cost.
- **Timestamping** — every record is tagged with an RTC or NTP-synced timestamp at the moment of capture, not at upload time, so the sequence of events is preserved even though upload happens hours later.
- **Queue-based upload** — records are stored as append-only lines (e.g., timestamp, sensor ID, value) and read back in the same order (FIFO) once connectivity returns, so the cloud sees data in the correct chronological order.
- **Retry with acknowledgement** — a record is only deleted from local storage after the server responds with an explicit ACK (HTTP 200 or MQTT QoS1 PUBACK); on failure, the device retries with exponential backoff rather than hammering the network immediately.
- **Overflow handling** — if local storage nears capacity during an unusually long outage, drop the oldest *low-priority* records first (FIFO with a priority tag) rather than newest data, since recent readings are usually more operationally relevant.

Q12. Designing for Five Years of Unattended Operation

Your IoT device must operate continuously for five years without human intervention. What changes would you make to both the hardware and software to improve long-term reliability?

Ans:

Five years without human intervention means the device must be able to detect its own problems and recover from most of them automatically, because a truck-roll to physically service the unit is often more expensive than the device itself.

Hardware changes

- **Industrial-grade components** — extended temperature range (−40°C to 85°C) parts and higher-MTBF passives instead of consumer-grade parts rated for indoor comfort conditions.
- **Corrosion protection** — conformal coating on the PCB, an IP65+ enclosure with proper gland seals, and a silica-gel desiccant sachet inside the enclosure to control condensation.
- **Redundant power path** — battery plus solar (or mains plus battery backup) so a single power source failure does not immediately take the device offline.
- **Derated components** — running capacitors, regulators, and connectors well below their maximum ratings extends their practical service life significantly.

Software changes

- **Watchdog timers** — both a hardware watchdog (independent of the CPU, catches total lockups) and software watchdogs on individual long-running tasks, with automatic reset on timeout.
- **OTA updates with rollback** — remote bug fixes and security patches without a site visit, with automatic rollback if a new image fails to boot correctly (detailed in Q17).
- **Persistent error logging** — a ring buffer in flash that survives reboots, capturing reset reason, last error codes, and sensor-fault flags for post-mortem analysis when the device is eventually serviced.
- **Self-diagnostics / health heartbeat** — periodic checks of supply voltage, sensor value sanity (range + rate-of-change checks), and a connectivity heartbeat sent to the cloud so a failing device is flagged for **predictive maintenance** before it fails completely, rather than being discovered dead during a routine check.

Q13. Automated vs Smart vs Intelligent Systems

Can an IoT system be considered "smart" if it does not use Artificial Intelligence? Justify your answer with suitable examples. What differentiates "smart" from "intelligent"?

Ans:

These three words get used interchangeably in marketing but describe genuinely different levels of capability, and the distinction matters for correctly scoping a project's claims.

Term	Defining trait	Example
Automated	Fixed rule, no adaptation, no connectivity required	A simple thermostat that switches the heater on/off at a fixed threshold; a timer-based sprinkler
Smart	Connected + remotely monitorable/controllable, may use conditional logic, but does not learn	A Wi-Fi thermostat with an app-editable schedule, or this assignment's BLE presence system reporting live status to a dashboard
Intelligent	Uses AI/ML to learn patterns and adapt behaviour autonomously over time	A learning thermostat that infers occupancy patterns and adjusts its own schedule without being told to

Can a system be "smart" without AI?

Yes. Connectivity and remote configurability are enough to earn the "smart" label - a smart plug that can be scheduled and toggled from a phone app uses no machine learning at all, but it is undeniably smart in the commonly understood sense. What it lacks is the ability to notice, on its own, that the schedule set by the user no longer matches actual usage and adjust itself — that step from "responds to instructions" to "forms its own instructions from observed data" is what separates smart from intelligent.

This distinction is directly relevant to grading and scoping expectations for a student IoT project: a BLE presence-monitoring system that logs and displays who is present is a legitimate **smart** system even with zero AI, and calling it "intelligent" would be an overclaim unless it also does something like predicting attendance patterns or flagging anomalous absence trends from historical data.

Q14. Polling vs Interrupt-Based Programming

Compare polling and interrupt-based programming in embedded systems. In which situations would interrupts significantly improve the performance of an IoT device?

Ans:

Polling and interrupt-based programming represent two fundamentally different approaches for handling events in embedded systems.

Aspect	Polling	Interrupt-driven
CPU usage	Wastes cycles repeatedly checking a flag/pin even when nothing has changed	CPU is free (or sleeping) until the event actually occurs
Latency	Bounded by the polling loop period — can miss fast events between checks	Near-immediate response to the triggering event
Power consumption	Higher — CPU stays active to keep polling	Lower — CPU can enter sleep and wake only on interrupt
Implementation complexity	Simple, predictable, easy to debug	Requires careful short-ISR design to avoid race conditions
Best suited for	Fixed-interval sampling where jitter does not matter (e.g., reading a slow ADC every 100 ms)	Time-critical or infrequent asynchronous events (button press, ultrasonic echo edge, rotary encoder pulse)

Interrupts make the biggest practical difference in exactly two situations, both of which showed up elsewhere in this assignment: **power-sensitive designs** (Q4), where the CPU can sleep between events and wake instantly on an interrupt rather than burning current in a polling loop; and **precise pulse-timing measurements** (Q10's ultrasonic sensor), where the echo pulse width must be captured accurately — a polling-based `pulseIn()`-style approach can miss the true edge timing if the loop is busy elsewhere, while an interrupt/timer-capture approach records the exact edge time regardless of what else the CPU is doing.

The trade-off is that interrupts are not free of cost either: too many high-frequency interrupts cause excessive context-switching overhead, and a poorly written ISR (one that does heavy work instead of just flagging an event and returning) can itself introduce jitter into other, more time-critical interrupts. So the practical rule is: use interrupts for infrequent, latency-sensitive, or power-sensitive events, and polling for steady, predictable, fixed-rate sampling where a slight delay does not matter.

Q15. Engineering Trade-offs in a BLE-Based Workforce Presence System

Every engineering design involves trade-offs. Looking at your own IoT project, identify three design decisions where you sacrificed one aspect (such as cost, speed, power, accuracy, or complexity) to improve another. Explain whether you believe those trade-offs were justified.

Ans:

Project: **BLE-Based Smart Workforce Presence and Availability Monitoring System using ESP32 and IoT Analytics.**

Three real trade-offs made during this design, each sacrificing one property to gain another:

#	Trade-off	Sacrificed	Gained	Justified?
1	BLE range (~10–30 m) chosen over Wi-Fi for badge-to-gateway link	Room-level positioning precision	Multi-month battery life on a coin-cell badge	Yes - the goal is presence/availability, not centimetre-level location
2	RSSI averaging/filtering done on the ESP32 gateway (edge) before upload, rather than sending raw RSSI to the cloud	Flexibility to re-run different smoothing algorithms later on raw historical data	Lower data volume and lower cloud/analytics cost	Yes - the dashboard only ever needed presence state, not raw signal history
3	Passive BLE advertisement scanning (no connection) instead of a connection-based GATT check-in	Guaranteed per-event delivery acknowledgement (occasional missed detection possible)	Scalability - gateway can passively see many badges at once with no per-device connection overhead, and lower badge power draw	Yes, with mitigation - - tuning the advertisement interval and using multiple gateway scan windows keeps missed-detection probability low for a system tolerant of a few seconds' latency

Looking at all three together, the common thread is that every trade-off favoured **scalability, power, and cost** over **precision and guaranteed delivery**. That is the correct bias for this specific project: a presence system that occasionally takes a few extra seconds to register someone, or cannot tell you which exact desk they are at, is still fully useful for its stated purpose (workforce availability), whereas a system that drained badge batteries in days or could not scale past a handful of badges would fail the actual use case regardless of how precise it was.

Q16. Local-Storage-and-Auto-Upload Flowchart

Create a flowchart for an IoT system that stores sensor readings locally whenever Wi-Fi is unavailable and automatically uploads the stored data once the connection is restored. Mention all the programs/protocols/I-O used in the system.

Ans:

This flowchart covers the same core problem as Q11 but at implementation detail: exactly what the firmware checks and does on every acquisition cycle, including which protocol/interface is used at each step.

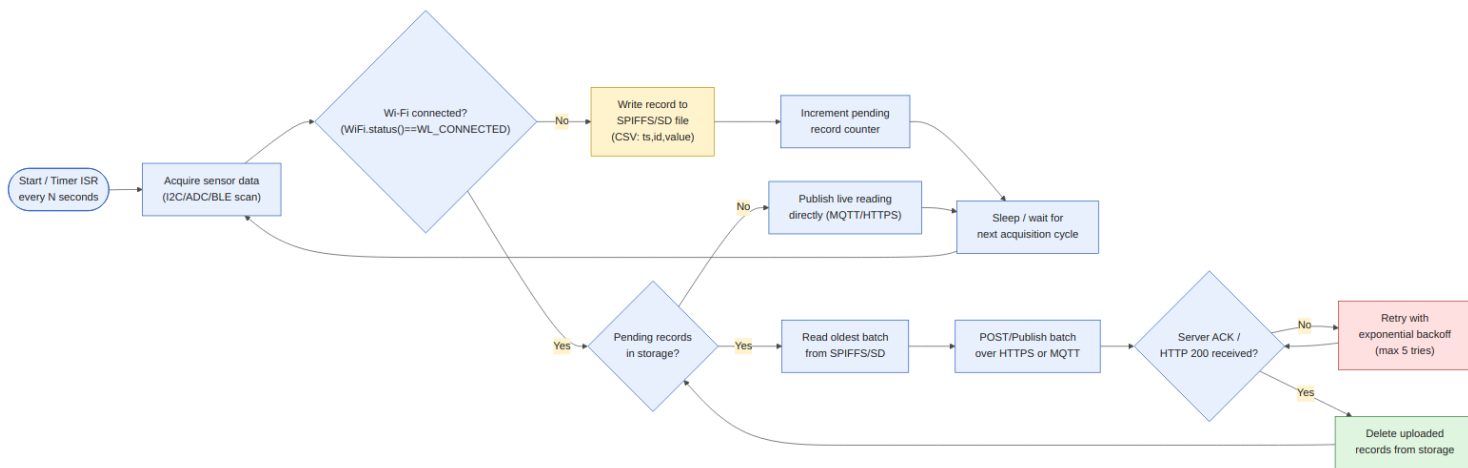


Figure Q16: Sensor acquisition, local storage, and confirmed-upload flowchart

Protocols and I/O interfaces used

Stage	Interface / Protocol
Sensor acquisition	I2C or ADC (GPIO), depending on sensor type
Wi-Fi status check	ESP32 WiFi.h stack — <code>WiFi.status() == WL_CONNECTED</code>
Local storage	SPI (for SD card) or internal flash (SPIFFS/LittleFS)
Data format	CSV or JSON line per record: timestamp, sensor ID, value
Upload	HTTPS POST or MQTT PUBLISH (QoS1) over Wi-Fi
Upload confirmation	HTTP 200 response, or MQTT PUBACK
Retry mechanism	Exponential backoff, capped at ~5 attempts before deferring to next cycle

The important detail that is easy to get wrong is the **order of delete-after-confirm**: records are only removed from SPIFFS/SD after the server has explicitly acknowledged receipt, never immediately after the upload call returns, since a network call can "succeed" locally (no exception thrown) while the server never actually received or processed the payload.

Q17. OTA (Over-the-Air) Firmware Update Flowchart with Rollback

Draw a flowchart showing how an ESP32 performs an OTA (Over-the-Air) firmware update, including error handling if the update fails.

Ans:

An OTA mechanism is only as trustworthy as its failure handling — the update path itself is straightforward, but a robust design spends most of its logic on what happens when any single step goes wrong.

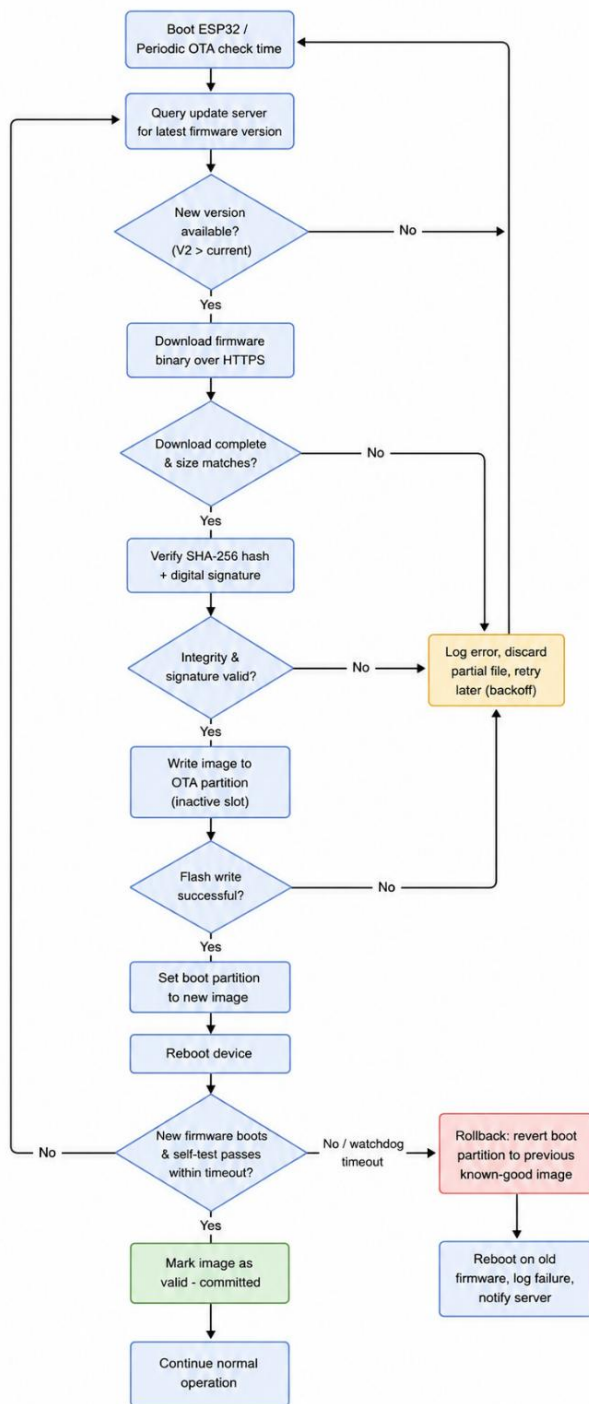


Figure Q17: OTA update flow with integrity verification, flashing, and rollback

Key stages

- **Version check** — device queries the update server (HTTPS) and compares the reported version against its own; proceeds only if a newer version is available.
- **Firmware download** — binary fetched over HTTPS; download size/completeness verified before proceeding, with a retry-with-backoff path if the transfer is incomplete.
- **Integrity verification** — SHA-256 hash and a digital signature (e.g., ECDSA) are checked before the image is trusted; a failed check is treated as a **security event**, not just a corrupted file, and is logged and reported.
- **Flashing** — the verified image is written to the **inactive** OTA partition (ESP32's dual-partition scheme, ota_0/ota_1), so the currently running firmware is never overwritten directly.
- **Boot and self-test** — the bootloader switches to the new partition and reboots; the new firmware must pass a self-test (successful Wi-Fi connect, sensor read, heartbeat to server) within a timeout window.
- **Rollback mechanism** — if the self-test fails or the watchdog times out, the bootloader automatically reverts the boot partition to the previous known-good image and reboots on it, logging the failure and notifying the server — this is what makes OTA safe to do unattended.

The dual-partition-plus-self-test-before-commit pattern (matching ESP-IDF's `esp_ota_*` APIs, including `esp_ota_mark_app_invalid_rollback_and_reboot()`) is what turns OTA from "a way to brick unreachable field devices" into a genuinely safe maintenance tool — without it, a bad update on a device five years into unattended field deployment (Q12) would mean a truck roll or a permanently dead unit.